LAB NO: 2 **Date:**

ADVANCED UNIX SHELL COMMANDS

Objectives:

1. To know the UNIX shell, special characters and commands.
2. To describe Unix commands.

1. Commands used to extract, sort, filter and process data

a. grep

grep is a command-line utility for searching plain-text data sets for lines matching a regular expression. grep was originally developed for the UNIX operating system, but is available today for all UNIX-like systems. Its name comes from the ed command g/re/p (globally search a regular expression and print), which has the same effect: doing a global search with the regular expression and printing all matching lines. Use grep –help to know the possible arguments for grep.

grep <someText> <fileName> #search, case sensitive, for <someText> in <file-name>, use -i for case insensitive search

grep -r <text> <folderName>/ # search for file names with occurrence of the text

With regular expressions:

grep -E ^<text> <fileName> #search start of lines with the word text

grep -E <0-4> <fileName> #shows lines containing numbers 0-4

grep –E <a-zA-Z> <fileName> # retrieve all lines with alphabetical letters.

Usage

Grep <word> <filename>

grep <word> file1 file2 file3

grep < word1> <word2> filename

cat <otherfile> | grep <word>

command | *grep* <something>

grep --color < word> <filename>

grep text * # * stands for all files in current directory

Examples:

$ *cat fruitlist.txt*

apple

apples
pineapple
fruit-apple
banana
pear
peach
orange

$ grep apple fruitlist.txt
apple
apples
pineapple
fruit-apple

$ grep -x apple fruitlist.txt # match whole line
apple

$ grep ^p fruitlist.txt
pineapple
pear
peach
$ grep -v apple fruitlist.txt #print unmatched lines
banana
pear
peach
orange

b. sort

sort is a program that prints the lines of its input or concatenation of all files listed in its argument list in sorted order. Sorting is done based on one or more sort keys extracted from each line of input. By default, the entire input is taken as sort key. Blank space is the default field separator. Note: Sort doesn't modify the input file content.

sort <any number of filenames> #sort the content of file(s)
sort <fileName> #sort alphabetically
sort -o <file> <outputFile> #write result to a file

sort -r <fileName> #sort in reverse order
sort -n <fileName> #sort numbers

c. wc (word count)

This shell command can be used to print the number of lines words in the input file/s.
$wc <fileName> #Number of lines, number of words, byte size of <fileName>.
Other arguments includes: -l (lines), -w (words), -c (byte size), -m
$ wc * : counts for all files in the current directory.

d. cut

cut is a data filter: it extracts columns from tabular data. If you supply the numbers of columns you want to extract from the input, cut prints only those columns on the standard output. Columns can be character positions or—relevant in this example—fields that are separated by TAB characters (default delimiter) or other delimiters.

Examples

1. *ls –l | cut –d "-" -f 2* #-d specifies the delimiter and default delimiter is tab. The permission for the group for the files can be obtained using this statement

cut –c 1-3 record.txt # -c specifies characters to be extracted
cut -c1,4,7 record.txt #characters 1, 4, and 7
cut -c1-3,8 record.txt #characters 1 thru 3, and 8
cut -c3- record.txt #characters 3 thru last
cut –f 1,4,7 record.txt #tab-separated fields 1, 4, and 7 #-f specifies fields to be extracted.

2. ls -l | tr -s ' ' | cut -d' ' -f5 # Lists files and prints only the size column. tr squeezes multiple spaces to one space.

e. sed (stream editor)

sed performs basic text transformations on an input stream (a file or input from a pipeline) in a single pass through the stream, so it is very efficient. However, it is sed's ability to filter text in a pipeline which particularly distinguishes it from other types of editor.

***sed* basics**

sed can be used at the command-line, or within a shell script, to edit a file non-interactively. Perhaps the most useful feature is to do a 'search-and-replace' for one string to another. You can embed your sed commands into the command-line that invokes

sed using the '-e' option, or put them in a separate file e.g. 'sed.in' and invoke sed using the '-f sed.in' option. This latter option is most used if the sed commands are complex and involve lots of regexps!

For instance*: sed -e 's/input/output/' my_file*

will echo every line from my_file to standard output, changing the first occurrence of 'input' on each line into 'output'. sed is line-oriented, so if you wish to change every occurrence on each line, then you need to make it a 'greedy' search & replace like so:

sed -e 's/input/output/g' my_file # g stands for global

By default the output is written to stdout. You may redirect this to a new file, or if you want to edit the existing file in place you should use the *-i* flag:

sed -e 's/input/output/' my_file > new_file

sed -i -e 's/input/output/' my_file

sed and regexps

What if one of the characters you wish to use in the search command is a special symbol, like / (e.g. in a filename) or * etc? Then you must escape the symbol just as for grep (and awk). Say you want to edit a shell scripts to refer to /usr/local/bin and not /bin any more, then you could do this

sed -e 's/\/bin/\/usr\/local\/bin/' my_script > new_script

What if you want to use a wildcard as part of your search – how do you write the output string? You need to use the special symbol '*&*' which corresponds to the pattern found. So say you want to take every line that starts with a number in your file and surround that number by parentheses:

sed -e 's/[0-9]/(&)/' my_file*

where [0-9] is a regexp range for all single digit numbers, and the '*' is a repeat count, means any number of digits.

Other _sed_ commands

The general form is *sed -e '/pattern/ command' my_file***,** where 'pattern' is a regexp and 'command' can be one of 's' = search & replace, or 'p' = print, or 'd' =delete, or 'i'=insert, or 'a'=append, etc. Note that the default action is to print all lines that do not match anyway, so if you want to suppress this you need to invoke sed with the '-n' flag and then you can use the 'p' command to control what is printed. So if you want to do a listing of all the subdirectories you could use below statement, as ls –l includes “d” at the start while listing the subdirectories.

ls -l | sed -n -e '/^d/ p'

Below statement, deletes all lines that start with the comment symbol '#' in my_file.

sed -e '/^#/ d' my_file

To insert a new line after a matching pattern is found the option “a” is used
sed –i ‘/word/a “xyz”’ filename
You can also use the range form
sed -e '1,100 command' my_file
to execute 'command' on lines 1,100. You can also use the special line number '$' to mean 'end of file'. For example below statement deletes all but the first 10 lines of a file.
sed -e '11,$ d' my_file

f. **tr (translate)**

The *tr* filter is used to translate one set of characters from the standard inputs to another.

Examples:

$tr “[a-z]” “[A-Z]” < filename #maps all lowercase characters in filename to uppercase. Content of the file is not changed.

tr 'abcd' 'jkmn' #maps all characters a to j, b to k, c to m, and d to n.

The character set may be abbreviated by using character ranges. The previous example could be written: *tr 'a-d' 'jkmn'*

The s flag (suppress) causes tr to compress sequences of identical adjacent characters in its output to a single token. For example,

tr -s '\n' #replaces sequences of one or more newline characters with a single newline.

The d flag causes tr to delete all tokens of the specified set of characters from its input. The *tr -d '\r'* statement removes carriage return characters.

The c flag indicates the complement of the first set of characters. The invocation *tr -cd '[:alnum:]'* therefore removes all non-alphanumeric characters.

2. Process management commands

a. ps

The *ps* command (short for "process status") displays the currently-running processes. *ps* command displays process id (PID), TTY (Terminal associated with the process), time The amount of CPU time used by the process and command name (CMD). For example:

$ *ps*

PID TTY TIME CMD

7431	pts/0	00:00:00	su
7434	pts/0	00:00:00	bash
18585	pts/0	00:00:00	ps

b. kill

kill is a command that is used in several popular operating systems to send signals to running processes in order to request the termination of the process. The signals in which users are generally most interested are SIGTERM and SIGKILL. The SIGTERM signal is sent to a process to request its termination. Unlike the SIGKILL signal, it can be caught and interpreted or ignored by the process. This allows the process to perform nice termination releasing resources and saving state if appropriate. The SIGKILL signal is sent to a process to cause it to terminate immediately (kill). This signal cannot be caught or ignored, and the receiving process cannot perform any clean-up upon receiving this signal.

Examples:

A process can be sent a SIGTERM signal in four ways (the process ID is '1234' in this case):

```
kill 1234
kill -s TERM 1234
kill -TERM 1234
kill -15 1234
```

The process can be sent a SIGKILL signal in three ways:

```
kill -s KILL 1234
kill -KILL 1234
kill -9 1234
```

3. File permission commands

a. chmod (Change mode)

The *chmod* numerical format accepts up to four octal digits. The three rightmost digits refer to permissions for the file owner, the group, and other users.

Numerical permissions

#	Permission	rwx
7	read, write and execute	rwx
6	read and write	rw-
5	read and execute	r-x

4	read only	r--
3	write and execute	-wx
2	write only	-w-
1	execute only	--x
0	none	---

The *chmod* command also accepts a finer-grained symbolic notation, which allows modifying specific modes while leaving other modes untouched. The symbolic mode is composed of three components, which are combined to form a single string of text:

$ chmod [references][operator][modes] file ...

The references (or classes) are used to distinguish the users to whom the permissions apply. If no references are specified it defaults to "all" but modifies only the permissions allowed by the umask. The references are represented by one or more of the following letters:

Reference	**Class**	**Description**
u	user	the owner of the file
g	group	users who are members of the file's group
o	others	users who are neither the owner of the file nor members of the file's group
a	all	all three of the above, same as ugo

The chmod program uses an operator to specify how the modes of a file should be adjusted. The following operators are accepted:

Operator	Description
+	adds the specified modes to the specified classes
-	removes the specified modes from the specified classes
=	the modes specified are to be made the exact modes for the specified classes

The modes indicate which permissions are to be granted or removed from the specified classes. There are three basic modes which correspond to the basic permissions:

Mode	Name	Description
r	read	read a file or list a directory's contents
w	write	write to a file or directory
x	execute	execute a file or recurse a directory tree

Command	Explanation
chmod a+r Comments.txt	read is added for all classes (i.e. User, Group and Others)
chmod +r Comments.txt	omitting the class defaults to all classes, but the resultant permissions are dependent on umask.
chmod a-x Comments.txt	execute permission is removed for all classes.
chmod a+rx viewer.sh	add read and execute for all classes.
chmod u=rw,g=r,o= Plan.txt	user(i.e. owner) can read and write, group can read, others cannot access.
chmod -R u+w,go-w docs	add write permissions to the directory docs and all its contents (i.e. recursively) for user and deny write access for everybody else.
chmod ug=rw groupAgreements.txt	user and group members can read and write (update the file).
chmod 664 global.txt	sets read and write and no execution access for the user and group, and read, no write, no execute for all others.

chmod 0744 myCV.txt	equivalent to u=rwx (400+200+100),go=r (40+ 4). The 0 specifies no special modes.

4. Other useful commands

a. echo

This is one of the most commonly and widely used built-in command, that typically used in scripting language and batch files to display a line of text/string on standard output or a file. This command writes its arguments to standard output. Example: echo this is OS lab manual, prints the input string on the terminal. It is not necessary to surround the strings with quotes, as it does not affect what is written on the screen. If quotes (either single or double) are used, they are not repeated on the screen.

b. bc (Basic Calculator)

After this command *bc* is started and it waits for your commands, example:

1. $bc (hit *enter*
key)5 + 2
7 #7 is response of bc i.e. addition of 5 + 2 you can even try
5 / 2
5 # to perform floating point operations use *bc -l*
5 > 2 #0 (Zero) is response of bc, How? Here it compare 5 with 2 as, Is 5 is greater than 2, (If I ask same-question to you, your answer will be YES)In UNIX (bc) gives this 'YES' answer by showing 0 (Zero) value.
2. echo "2^5" | bc #32
3. echo "scale=2; 5 / 2" | bc #output 2.50
4. echo "5 / 2" |bc #2
5. echo "5 / 2" |bc -l # 2.50000000000000000000

The vi editor

The *vi* editor is a visual editor used to create and edit text, files, documents and programs. It displays the content of files on the screen and allows a user to add, delete or change part of text. There are three modes available in the *vi* editor, they are

i. Command mode
ii. Input (or) insert mode.

Starting vi :

The *vi* editor is invoked by giving the following commands in UNIX prompt.

Syntax : *$vi <filename> (or) $vi*

This command would open a display screen with 25 lines and with tilt (~) symbol at the start of each line. The first syntax would save the file in the filename mentioned and for the next the filename must be mentioned at the end.

Options :

1.*vi +n <filename>* - this would point at the nth line (cursor pos).

2.*vi –n <filename>* - This command is to make the file to read only to change from one mode to another press escape key.

Saving and Quitting from vi

To move editor from command node to edit mode, you have to press the <ESC> key.

<ESC> w Command	To save the given text present in the file.
<ESC> q! Command	To quit the given text without saving.
<ESC> wq Command	This command quits the vi editor after saving the text in the mentioned file.
<ESC> x Command	This command is same as "wq" command it saves and quit.
<ESC> q Command	This command would quit the window but it would ask for again to save the file.

Lab Exercises

1. Execute all the commands explained in this section and write the output.
2. Write grep commands to do the following activities:
   - To select the lines from a file that have exactly two characters.
   - To select the lines from a file that start with the upper case letter.
   - To select the lines from a file that end with a period.
   - To select the lines in a file that has one or more blank spaces.
   - To select the lines in a file and direct them to another file which has digits as one of the characters in that line.
3. Create file studentInformation.txt using *vi* editor which contains details in the following format.

RegistrationNo:Name:Department:Branch:Section:Sub1:Sub2:Sub3

1234:XYZ:ICT:CCE:A:80:60:70 … (add atleast 10 rows)

i) Display the number students(only count) belonging to ICT department.

ii) Replace all occurrences of IT branch with "Information Technology" and save the output to ITStudents.txt

iii) Display the average marks of student with the given registration number "1234" (or any specific existing number).